

Deployment Guide and QA Testing Guidelines

[Deployment Guide & QA Testing Guidelines](#)

- [Revision History](#)
- [1. Introduction](#)
- [2. CI/CD Overview](#)
- [3. Docker Standards](#)
- [4. Kubernetes Deployment Standards](#)
- [5. GitHub Actions Workflows](#)
- [6. Jenkins Deployment Orchestration](#)
- [7. Release Process](#)
- [7A. Environment Management](#)
- [7B. Incident Response for Production Issues](#)
- [8. Testing Strategy](#)
- [9. Bug Reporting](#)
- [10. Test Case Standards](#)
- [11. Internal Contacts](#)
- [12. Related Documents](#)
- [13. References](#)

Deployment Guide & QA Testing Guidelines

Series Tech Ltd. — Internal Use Only

[Company Logo Placeholder — SeriesTech_Logo.png]

Classification: Internal — Confidential **Document Number:** STL-DEVQA-DEP-008 **Version:** 4.6 **Last Updated:** 2026-06-20 **Owner:** Head of DevOps **Approver:** VP of Engineering

Revision History

Version	Date	Author	Summary of Changes
1.0	2022-08-14	J. Thomas (DevOps)	Initial CI/CD and deployment guide
2.0	2023-11-02	J. Thomas (DevOps)	Migrated pipelines from Jenkins-only to GitHub Actions + Jenkins hybrid
3.0	2024-12-19	P. Nambiar (QA)	Merged QA Testing Guidelines into single document
4.0	2025-10-07	P. Nambiar (QA)	Added Test Case Standards and Bug Reporting workflow
4.6	2026-06-20	J. Thomas (DevOps)	Updated Kubernetes deployment checklist for multi-region rollout

1. Introduction

This guide combines DevOps deployment procedures with QA testing standards into a single reference used across every release of SeriesTech products, including Project Alpha. It is maintained jointly by the DevOps and QA departments.

1.1 Scope

Applies to all services deployed to Staging and Production Kubernetes clusters, including Project Alpha's Ingestion, Retrieval, Reranking, and Generation services described in the Project Alpha Technical Design Document (STL-ALPHA-TDD-005).

2. CI/CD Overview

2.1 Pipeline Tools

SeriesTech uses **GitHub Actions** for build, lint, and test stages, and **Jenkins** for deployment orchestration to Kubernetes clusters, retained for its mature integration with the internal Change Advisory Board (CAB) approval workflow.

2.2 Pipeline Stages

```
Code pushed to feature branch
↓
GitHub Actions: lint, unit tests, security scan (SAST)
↓
PR merged to develop (per Engineering Standards, STL-ENG-STD-003, Section 6)
↓
GitHub Actions: integration tests against Staging-mirrored environment
↓
Jenkins: deployment to Staging cluster
↓
QA: manual and automated regression testing on Staging
↓
Jenkins: deployment to Production cluster (post CAB approval)
```

3. Docker Standards

3.1 Image Build Requirements

- All services must use multi-stage Dockerfiles to minimize final image size.
- Base images must come from the approved internal registry (`registry.seriestech.com/base-images/`), never public Docker Hub images directly, to ensure vulnerability scanning compliance.
- Images must be tagged with the Git commit SHA, never `latest` , for production deployments.

3.2 Example Dockerfile Pattern

```
FROM registry.seriestech.com/base-images/python:3.11-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

FROM registry.seriestech.com/base-images/python:3.11-slim
WORKDIR /app
COPY --from=builder /usr/local/lib/python3.11/site-packages /usr/local/lib/python3.11/site-packages
```

```
COPY . .
CMD ["gunicorn", "app.wsgi:application", "--bind", "0.0.0.0:8000"]
```

4. Kubernetes Deployment Standards

4.1 Namespace Strategy

Environment	Namespace Pattern	Example
Dev	dev-<service>	dev-alpha-retrieval
Staging	staging-<service>	staging-alpha-retrieval
Production	prod-<service>-<region>	prod-alpha-retrieval-apse1

4.2 Deployment Manifests

All Kubernetes manifests are managed via Helm charts stored in the `st-helm-charts` repository, with environment-specific values files (`values-dev.yaml` , `values-staging.yaml` , `values-prod.yaml`).

4.3 Resource and Scaling Configuration

Service Type	CPU Request	Memory Request	HPA Min/Max Replicas
API services (Django/Spring Boot)	500m	512Mi	2 / 10
Ingestion Service	1000m	2Gi	1 / 6
Retrieval Service	1000m	2Gi	2 / 12
Reranking Service	2000m	4Gi (GPU-backed nodes)	1 / 4

5. GitHub Actions Workflows

5.1 Standard Workflow Jobs

Job	Trigger	Purpose
lint	Every push	Runs ESLint/ruff/checkstyle per Engineering Standards
unit-test	Every push	Runs unit test suite, enforces 80% coverage gate
security-scan	Every push	Runs SAST (Semgrep) and dependency vulnerability scan
integration-test	PR to develop	Runs integration suite against ephemeral environment
build-and-push	Merge to develop/main	Builds Docker image, pushes to internal registry

6. Jenkins Deployment Orchestration

6.1 Deployment Pipeline Stages

```
Jenkins job triggered post GitHub Actions build-and-push
↓
```

Change Advisory Board (CAB) approval gate for Production deployments

↓

Helm upgrade applied to target namespace

↓

Automated smoke tests run against deployed pods

↓

Deployment marked successful or automatically rolled back

6.2 Rollback Policy

If smoke tests fail, or error rates exceed 2% within 10 minutes of deployment, Jenkins automatically triggers a Helm rollback to the previous stable release and notifies the on-call DevOps engineer via PagerDuty.

7. Release Process

7.1 Release Cadence

SeriesTech follows a bi-weekly release train for platform services, with hotfixes released outside the train as needed following the `hotfix/` branch strategy defined in Engineering Standards (STL-ENG-STD-003), Section 5.

7.2 Release Checklist (Deployment Checklist)

Step	Owner	Status Gate
Code freeze on <code>release/<version></code> branch	Tech Lead	All planned PRs merged
Full regression suite executed	QA	Zero P1/P2 defects open
Staging sign-off	QA Lead	Formal sign-off in Jira
CAB approval	Release Manager	Documented approval
Production deployment via Jenkins	DevOps	Smoke tests pass
Post-release monitoring (24 hours)	DevOps + QA	No anomalous error rates
Release notes published	Tech Lead	Posted to Confluence

7A. Environment Management

7A.1 Environment Matrix

Environment	Refresh Cadence	Data Source	Access
Dev	On-demand	Synthetic/seed data	All Engineering
Staging	Nightly refresh from anonymized Production snapshot	Anonymized Production copy	Engineering, QA, DevOps
Production	Continuous	Live client data	DevOps, Architecture (break-glass only)

7A.2 RACI Matrix — Release Activities

Activity	DevOps	QA	Tech Lead	Release Manager
Code freeze	I	I	R	A
Regression testing	I	R	C	A
Staging deployment	D	I	I	A

Staging deployment	R	I	I	A
CAB submission	C	I	C	R
Production deployment	R	I	I	A
Post-release monitoring	R	C	I	A

(R = Responsible, A = Accountable, C = Consulted, I = Informed)

7B. Incident Response for Production Issues

7B.1 On-Call Rotation

DevOps maintains a weekly on-call rotation managed through PagerDuty, with a primary and secondary on-call engineer for each major service group (Platform APIs, Project Alpha AI Services, Data Infrastructure).

7B.2 Production Incident Workflow

```

Alert fires in Prometheus/Grafana or PagerDuty
↓
On-call engineer acknowledges within SLA (15 minutes for P1)
↓
Incident channel created in Microsoft Teams, stakeholders paged
↓
Mitigation applied (rollback, scale-up, failover)
↓
Post-incident review scheduled within 3 business days

```

Best Practice: Every P1 or P2 production incident should result in a blameless post-incident review documenting root cause and at least one concrete follow-up action, tracked in Jira under the `INCIDENT-FOLLOWUP` label.

8. Testing Strategy

8.1 Test Pyramid

SeriesTech follows a test pyramid approach: a large base of unit tests, a moderate layer of integration tests, and a smaller set of end-to-end tests, avoiding over-reliance on slow, brittle UI automation.

8.2 Testing Matrix

Test Type	Owner	Frequency	Tooling
Unit Tests	Engineering	Every commit	pytest, JUnit, Jest
Integration Tests	Engineering + QA	Every PR to <code>develop</code>	pytest + testcontainers
Regression Tests	QA	Every release	Selenium, Playwright
Performance Tests	QA + DevOps	Before major releases	Locust, k6
RAG Quality Evaluation	QA + Architecture	Weekly (Project Alpha)	Custom eval harness against golden Q&A set

8.3 RAG-Specific Testing (Project Alpha)

Given Project Alpha’s retrieval-augmented generation pipeline, QA maintains a golden dataset of question-answer-citation triples to evaluate retrieval precision, answer faithfulness, and citation accuracy after any change to chunking, embedding models, or reranking logic.

9. Bug Reporting

9.1 Bug Severity Definitions

Severity	Definition	Example
P1	Production down or data loss	Retrieval Service crash affecting all tenants
P2	Major functionality broken, workaround difficult	Citations missing from all generated answers
P3	Minor functionality broken, workaround available	Incorrect page number in citation
P4	Cosmetic or low-impact	Misaligned button in Admin Portal

9.2 Bug Report Workflow

```
graph TD
    A[Tester or user identifies defect] --> B[Bug logged in Jira with severity, steps to reproduce, environment]
    B --> C[Triage by QA Lead and Tech Lead, priority assigned]
    C --> D[Assigned to Engineer for fix]
    D --> E[Fix verified by QA, bug closed]
```

10. Test Case Standards

10.1 Required Fields for Test Cases

Every test case documented in the Test Management system must include: Test ID, Title, Preconditions, Steps, Expected Result, Actual Result, and Status (Pass/Fail/Blocked).

10.2 Example Test Case

Example — Test ID QA-ALPHA-0231: Verify that a query with no relevant indexed content returns a “No relevant information found” message rather than a hallucinated answer, for a tenant with only HR documents indexed, querying “What is our Kubernetes rollback policy?”

11. Internal Contacts

Purpose	Contact
DevOps On-Call	devops-oncall@seriestech.com
QA Lead	qa-lead@seriestech.com
Release Management	release-mgmt@seriestech.com
CAB Submissions	cab-approvals@seriestech.com

12. Related Documents

- STL-ENG-STD-003 — Engineering Standards
- STL-ALPHA-TDD-005 — Project Alpha Technical Design Document

- STL-ARCH-SYS-009 — Enterprise System Architecture & FAQ

13. References

- SeriesTech CI/CD Pipeline Reference, DevOps internal wiki, 2026
 - Change Advisory Board Charter, v3
 - RAG Evaluation Harness Documentation, QA internal wiki
-

Internal Use Only — Series Tech Ltd. © 2026. Unauthorized distribution of this document outside the organization is prohibited.